

Lernziele:

- Vertiefung: Pointer!

Zielsetzung

Unser Compiler kann aktuell einfache Ausdrücke verarbeiten, die Variablen enthalten. Eine Variable muss nicht deklariert werden und ist stets eine globale Variable (es gibt also zu jeder Variable ein zugehöriges Label im Datensegment).

In dieser Session sollen unäre Operatoren (also Operatoren mit nur einem Operanden) unterstützt werden. Ein weniger interessanter unärer Operator ist das unäre Minus zur Negation einer Zahl. Diesen verschieben wir ans Ende. Spannender sind der Adressoperator `&` und der Dereferenzierungsoperator `*`. Diese Operatoren zu unterstützen ist nicht nur interessant, sondern fördert auch das Verständnis für Pointer im Allgemeinen.

Schritt 1 (Lexer anpassen)

Aktuell erkennt der Lexer das Zeichen `&` noch nicht. Testet dies, indem ihr `./xtest_lexer` ausführt und ein `&` eingibt. In der Ausgabe sollte dafür ein `BAD_TOKEN` auftauchen. Den Lexer zu erweitern ist nicht schwer und eine gute Übung, um die Struktur des Compiler-Projekts im Überblick zu behalten. Insbesondere zählt es sich aus, dass man den Lexer unabhängig vom Rest erweitern und testen kann.

Aufgabe

Deklariert für `TokenKind` eine neue Enum-Konstante `AMPERSAND` und passt die Implementierung im Lexer so an, dass `&` erkannt wird:

```
MCL:s24 lehn$ ./xtest_lexer
&
1.1: AMPERSAND (&)
```

Schritt 2 (Datenstruktur für Expression-Trees)

Bei den Expressions haben wir aktuell zwei Knotenarten:

- **Primary-Nodes:** haben keine Child-Nodes
- **Binary-Nodes:** haben zwei Child-Nodes

Was wir brauchen, ist eine neue Knoten-Art "Unary-Node" mit genau einem Child-Node. Für diese Art gibt es zwei Ausprägungen:

- `EXPR_ADDR` für Knoten, die der Adressoperator liefern wird.
- `EXPR_DEREF` für Knoten, die der Dereferenzierungsoperator liefern wird.

Meine Anpassung von `ExprKind` seht ihr rechts und könnt ihr entsprechend übernehmen.

```
enum ExprKind
{
    EXPR_BINARY,
    EXPR_ADD = EXPR_BINARY,
    EXPR_ASSIGN,
    EXPR_DIV,
    EXPR_MUL,
    EXPR_SUB,
    EXPR_BINARY_END,

    EXPR_UNARY = EXPR_BINARY_END,
    EXPR_DEREF = EXPR_UNARY,
    EXPR_ADDR,
    EXPR_UNARY_END,

    EXPR_PRIMARY = EXPR_UNARY_END,
    EXPR_INTEGER = EXPR_PRIMARY,
    EXPR_IDENTIFIER,
    EXPR_PRIMARY_END,
};
```

Mein Struct `Expr` habe ich so angepasst, dass Unary-Nodes ein eigenes Member `child` bekommen, das einfach ein Zeiger auf den Child-Node ist:

```

struct Expr
{
    kind:      ExprKind;
    decimal:   -> UStr;
    identifier: -> UStr;
    left, right: -> Expr;
    child:     -> Expr;
};

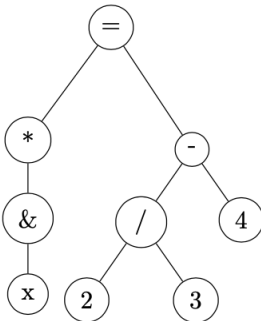
```

Zur Erinnerung, der Datentyp `Expr` ist ein sogenannter algebraischer Datentyp (oder "zusammengesetzter Datentyp"). Konkret hat das folgende Bedeutung:

- Hat `kind` den Wert `EXPR_INTEGER`, dann hat das Member `decimal` einen sinnvollen Wert (ein Zeiger auf die String-Darstellung des Integer).
- Hat `kind` den Wert `EXPR_IDENTIFIER`, dann hat das Member `identifier` einen sinnvollen Wert (ein Zeiger auf die String-Darstellung der Variable).
- Hat `kind` den Wert einer Binary-Expression (also ein Wert von `EXPR_ADD` bis `EXPR_SUB`), dann haben die Member `left` und `right` sinnvolle Werte (Zeiger auf die zwei Child-Nodes).
- Hat `kind` den Wert einer Unary-Expression (also `EXPR_ADDR` oder `EXPR_DEREF`), dann hat das Member `child` einen sinnvollen Wert (Zeiger auf den Child-Node).

Spezielle Enum-Konstanten wie `EXPR_BINARY`, `EXPR_BINARY_END`, etc. haben den Zweck, dass man in der Implementierung die notwendigen Fallunterscheidungen einfacher handhaben kann. Es macht es auch leichter, neue Knoten-Arten und neue Ausprägungen in der Implementierung zu unterstützen.

In diesem Schritt werden wir die Implementierung zunächst nur so anpassen, dass für die neuen Knoten-Arten wieder eine LaTeX-Darstellung unterstützt wird. Die Generierung von Assembler-Code verschieben wir auf den nächsten Schritt. Das erste Ziel ist es, dass zum Beispiel so ein Bild erzeugt werden kann:



Passt dazu zunächst das Testprogramm `xtest_expr.abc` an:

1. Die Zeile

```
genReleaseReg(loadExpr(expr));
```

solltet ihr zunächst auskommentieren, da damit versucht wird, Assembler-Code für den Ausdruck zu erzeugen.

- In den bisherigen Code können jetzt Funktionsaufrufe eingefügt werden, die die neuen Knotenarten erzeugen. Ich habe dazu Funktionen `createDerefExpr` und `createAddrExpr` geschrieben, die jeweils einen Zeiger auf einen Expression-Node erwarten und einen Zeiger auf einen neu generierten Knoten zurückgeben. In `expr.hdr` sind diese wie folgt deklariert:

```

extern fn createAddrExpr(child: -> Expr): -> Expr;
extern fn createDerefExpr(child: -> Expr): -> Expr;

```

Mein Testprogramm `xtest_expr.abc` sieht so aus:

```
@ <stdio.h>
@ "expr.hdr"
@ "gen.hdr"

fn main()
{
    local expr: -> Expr;

    local _2: -> UStr = UStrCreate("2");
    local _3: -> UStr = UStrCreate("3");
    local _4: -> UStr = UStrCreate("4");
    local _x: -> UStr = UStrCreate("x");

    expr = createBinaryExpr(EXPR_SUB,
                            createBinaryExpr(EXPR_DIV,
                                                createIntegerExpr(_2),
                                                createIntegerExpr(_3)),
                            createIntegerExpr(_4));
    printExprLatexTree(expr);
    expr = createBinaryExpr(EXPR_ASSIGN,
                            createDerefExpr(
                                createAddrExpr(
                                    createIdentifierExpr(_x))),
                            expr);
    printExprLatexTree(expr);
    printf("\n");

    //genReleaseReg(loadExpr(expr));

    releaseExpr(expr);
}
```

Entscheidend ist, dass die neuen Funktionen zum Generieren der Unary-Nodes mindestens einmal aufgerufen werden. Bei mir ist das der geschachtelte Aufruf

```
createDerefExpr(createAddrExpr(createIdentifierExpr(_x)))
```

mit dem zuerst ein Knoten für eine Variable erzeugt wird. Dieser Knoten wird dann zum Child-Node eines Adressoperators und dieser Knoten wiederum zum Child-Node eines Dereferenzierungsoperators.

Aufgabe

1. Deklariert die Funktionen zum Erzeugen der neuen Knotenarten und passt den Test so an, dass sie im Testprogramm verwendet werden. Stellt sicher, dass sich alles ohne Fehler übersetzen lässt und nur noch ein Linker-Fehler auftaucht.
2. Implementiert als nächstes die Funktionen zum Erzeugen der neuen Knoten in `expr.abc`. Das Testprogramm sollte sich jetzt zu einem ausführbaren Programm übersetzen lassen. Wenn man das Programm aber ausführt, werden die neuen Knoten allerdings noch ignoriert ("verschluckt").
3. Passt die Funktion `printExprTree` so an, dass die Unary-Nodes nun auch tatsächlich ausgegeben werden. Hier ist zu beachten, dass man im LaTeX-Code das &-Zeichen mit einem Backslash escapen muss. Und um einen Backslash mit `printf` ausgeben zu können, muss man den Backslash im Formatstring mit einem Backslash escapen (also zwei Backslashes verwenden):
 - Zunächst soll immer ein `[` ausgegeben werden.
 - Falls es ein Adressknoten ist, soll dann `\&` (also Backslash gefolgt von `&`) und falls es ein Dereferenzierungsknoten ist, ein `*` ausgegeben werden.
 - Dann soll ein `]` ausgegeben werden.

Schritt 3 (Code-Generierung für Expression-Trees)

In diesem Schritt soll nun auch Assembler-Code für die neuen Knotentypen erzeugt werden. Hier kann man so vorgehen, dass man zunächst im Testprogramm `xtest_expr.abc` die Anweisung `genReleaseReg(loadExpr(expr));`

wieder aktiviert, also den Kommentar entfernt, und dann schaut, an welchen Stellen Assertion-Failures bei der Ausführung des Tests auftauchen. Diese Vorgehensweise setzt natürlich voraus, dass bisher in `expr.abc` konsequent mit Assertions gearbeitet wurde. Das ist bei uns der Fall, da die Implementierung nach dem sogenannten "Design by Contract"-Prinzip arbeitet. Jede Funktion wird als eine Art Vertrag aufgefasst. Erfüllen die Parameter eine vereinbarte Vorbedingung (zum Beispiel bei Expressions das Zusammenspiel von `kind` und den entsprechenden Members), dann wird zugesichert, dass ein Auftrag erfüllt wird (zum Beispiel die Ausgabe von LaTeX-Code oder Assembler-Code) oder ein Assertion-Failure ausgelöst wird.

Aufgabe

1. Übersetzt das Testprogramm und führt es aus. Dabei sollte in der Funktion `loadExprAddress` ein Assertion-Failure ausgelöst werden:

```
Assertion 'expr->kind == EXPR_IDENTIFIER' failed.
```

Die Aufgabe der Funktion ist es, Assembler-Code zu generieren, mit dem die Adresse des Ausdrucks in ein Register geladen wird. Das ist natürlich nicht für jede Art von Ausdruck möglich. Zum Beispiel hat eine Konstante wie `42` oder eine Summe wie `x + 2` keine Adresse. Adressen haben nur Variablen, und diese sind bisher nur "direkt" vorgekommen, wenn die Expression eben ein Primary-Knoten für einen Identifier war. Jetzt gibt es aber einen weiteren Fall für einen Expression-Knoten, der durchaus eine Adresse hat, nämlich jeden Dereferenzierungsknoten.

- Zum Beispiel verweist `*x` auf die Speicherfläche einer Variable, wobei die Adresse der Variable der Wert von `x` ist. In diesem Fall sollte die Funktion also Assembler-Code erzeugen, um den Wert von `x` in ein Register zu laden.
- Auch Ausdrücke wie `**x` oder `&x` verweisen auf Speicherflächen. Die Adresse der Speicherfläche von `**x` ist `*x`, die Adresse der Speicherfläche von `&x` ist `&x`.

Alle Fälle eines Dereferenzierungsknotens haben gemeinsam, dass die Adresse des Knotens der Wert des Child-Nodes ist. Die Funktion kann man also wie folgt programmieren:

- Hat `kind` den Wert `EXPR_IDENTIFIER`, dann gibt man wie bisher `genLoadAddress(expr->identifier)` zurück
- Hat `kind` den Wert `EXPR_DEREF`, dann gibt man `loadExpr(expr->child)` zurück, denn damit wird Assembler-Code erzeugt, um den Wert des Child-Nodes in ein Register zu laden.
- In allen anderen Fällen löst man ein Assertion-Failure aus, da die Voraussetzung "Knoten hat eine Adresse" für den Vertrag nicht erfüllt ist. Diesen Vertrag kann man natürlich in Form eines Kommentars vor der Deklaration von `loadExprAddress` festhalten. Aber eigentlich ist das bereits implizit durch den sprechenden Namen `loadExprAddress` schon dokumentiert. Kommentare im Source-File (nicht im Header-File) sind in der Regel ein Zeichen von schlechtem Design und sollten nur selten notwendig sein (siehe Linux Kernel Coding Style).

Passt die Implementierung von `loadExprAddress` an. Das Programm sollte übersetzen. Bei der Ausführung wird wieder ein Assertion-Failure ausgelöst. Aber an einer anderen Stelle.

2. Der neue Assertion-Failure sieht bei mir so aus:

```
Assertion '0 && "loadExpr: not all cases handled!"' failed.
```

Ausgelöst wird dies in der Funktion `loadExpr`. Hier ist der Grund offensichtlich, wenn man sich den Code anschaut. Bisher wird dort nur zwischen Binary- und Primary-Nodes unterschieden. Was man braucht, ist also Code, um auch Unary-Nodes zu behandeln:

1. Hat `kind` den Wert `EXPR_ADDR`, dann stellt der Knoten einen Adressoperator dar. Es sollte Code erzeugt werden, um die Adresse des Child-Nodes in ein Register zu laden. Das ist einfach, man gibt einfach `loadExprAddress(expr->child)` zurück.
2. Hat `kind` den Wert `EXPR_DEREF`, dann stellt der Knoten einen Dereferenzierungsoperator dar. Auch der Fall ist nicht schwer. Zunächst kann man mit `loadExpr(expr->child)` Code erzeugen, um die Adresse des Child-Nodes in ein Register zu laden. Dieses Register kann man dann mit `genReleaseReg` wieder zur Verfügung stellen, sodass es in einem anschließenden Aufruf von `genFetch` wiederverwendet werden kann.

Passt die Funktion `loadExpr` entsprechend an. Danach sollte das Testprogramm gültigen Assembler-Code ausgeben. Prüft, dass der Assembler-Code den Wert des Ausdrucks am Ende in Register `%3` speichert.

Schritt 4 (Parser anpassen)

Jetzt muss noch die Grammatik angepasst werden, damit der Adressoperator und der Dereferenzierungsoperator in unserer Programmiersprache verwendet werden können:

```
prog = { statement }
statement = expr ";"
expr = additive { "=" expr }
additive = term { ( "+" | "-" ) term }
term = unary { ( "*" | "/" ) unary }
unary = (( "*" | "&" ) unary)
        | factor
factor = decimal-literal | identifier | "(" expr ")"
```

Neu ist die Produktionsregel `unary`, die hier rekursiv formuliert wurde, um auch die Mehrfachverwendung des Dereferenzierungsoperators oder Adressoperators zuzulassen (wie zum Beispiel `***x`). Alternativ kann man die Regel aber auch iterativ mit

```
unary = {( "*" | "&" )} factor
```

beschreiben.

Aufgabe

Implementiert die Parse-Funktion `parseUnary` (die rekursive oder iterative Variante):

```
fn parseUnary(): -> Expr
{
    // ...
}
```

Passt die Implementierung der Parse-Funktion `parseTerm` an, sodass diese jetzt `parseUnary` statt `parseFactor` aufruft.

Wenn alles klappt, sollte der Compiler `xtest_abc` jetzt auch diesen Code verarbeiten können:

```
MCL:s24 lehn$ cat test
x = 5;
y = &x;
z = &y;
**z = 42;
x;
```

Wenn man dies übersetzt und ausführt, sollte der Wert von `x` in der letzten Anweisung `42` sein. Und dies sollte auch der Exit-Code sein:

```
MCL:s24 lehn$ ./xtest_abc < test > foo.s
MCL:s24 lehn$ ulmas foo.s
MCL:s24 lehn$ ulm a.out
MCL:s24 lehn$ echo $?
42
```

Schritt 5 (Spielerei mit dem Compiler)

Unser Compiler unterstützt auch Pointer-Arithmetik, wie in diesem Beispiel:

```
MCL:s24 lehn$ cat test2
x = 5;
y = 42;
z = &y;
*(z + 8) = 13;
x;
```

Hier wird der Variable `z` zunächst die Adresse von `y` zugewiesen. In der Zuweisung

```
*(z + 8) = 13;
```

wird der Wert von `z` um `8` erhöht und an diese Adresse der Wert `13` geschrieben. Schaut man sich das Datensegment des generierten Assembler-Codes an, dann sieht man, dass dort die Variable `x` steht. Deshalb ist der Exit-Code `13` (der Wert von `x`):

```
MCL:s24 lehn$ ./xtest_abc < test2 > foo.s && ulmas foo.s && ulm a.out; echo $?
13
```

Aufgabe: Führt das Programm im Debugger `udb-tui` aus, um das zu beobachten. Was muss man ändern, dass `y` anstelle von `x` überschrieben wird?

Man kann auch verrückte Dinge machen und an eine hart kodierte Adresse einen Wert schreiben:

```
MCL:s24 lehn$ cat test3
*80 = 8196;
*80 = *80 * 256 * 256 * 256 * 256 * 256 * 16;
123;
```

In diesem Fall wird an die Adresse `80` zunächst der Wert `8196` geschrieben. Damit wird das Text-Segment des Programms überschrieben, das heißt, das Programm modifiziert sich bei der Ausführung selbst!

Aufgabe: Führt das Programm im Debugger aus und erklärt eurem Nachbarn, wieso das Programm den Exit-Code `80` und nicht `123` hat.

Schritt 6 (Fehlerbehandlung)

Es ist lustig bis schön, dass unsere Programmiersprache ein paar verrückte Dinge erlaubt, wie das Dereferenzieren einer Zahl (also einer hart codierten Adresse). Alles, was technisch möglich ist, wird unterstützt.

Aber unschön und wenig witzig ist, dass illegaler Code zu einer Assertion statt zu einer Fehlermeldung führt. Zum Beispiel dieses Programm, in dem der Adressoperator auf eine Zahl angewandt wird:

```
MCL:s24 lehn$ ./xtest_abc
&5;
expr.abc:120: Assertion '0 && "expression has no address"'
failed.
Abort trap: 6
```

Hier sollte eine Fehlermeldung anzeigen, in welcher Zeile und Spalte versucht wird, den Adressoperator auf einen Ausdruck anzuwenden, der keine Adresse hat. Um das zu ermöglichen,

kann man in `expr.hdr` eine Funktion deklarieren, um abfragen zu können, ob ein Ausdruck eine Adresse hat. Deklariert in `expr.hdr` die Funktion:

```
extern fn addressableExpr(expr: ->Expr): bool;
```

Im Parser kann man die Fehlerbehandlung in dieser Form in `parseUnary` (rekursive Variante) zum Beispiel so einbauen:

```
if (token.kind == AMPERSAND) {  
    getToken();  
    local expr: -> Expr = parseUnary();  
    if (!addressableExpr(expr)) {  
        printf("%u.%u: expression has no address\n",  
            token.pos.row, token.pos.col);  
        return nullptr;  
    }  
    return createAddrExpr(parseUnary());  
}
```

Aufgabe

Implementiert die Funktion `addressableExpr`. Damit sollte man Fehlermeldungen wie die folgende erhalten:

```
MCL:s24 lehn$ ./xtest_abc  
&12;  
1.4: expression has no address  
1.4: syntax error
```